

QA Automation Framework for Myntra Using Selenium, Pytest and BDD

Project Overview

This project is an automation testing framework developed for the Myntra e-commerce website using Python, Selenium WebDriver, Pytest, and BDD with Behave. The main objective of the project is to automate important user workflows on myntra for product searching to product to login during order placement.

The project follows the Page Object Model (POM) architecture where separate classes are created for each webpage such as HomePage, ProductPage, CartPage, and LoginPage. Reusable utility modules were also implemented for logging, screenshots, configuration handling, and CSV-based test data execution. Both Selenium Pytest testing and BDD Behave testing implemented for same flow as different projects.

Test Strategy and Problem Definition

Modern e-commerce websites contain highly dynamic user interfaces with filters, dropdowns, category navigation, login systems, and checkout flows. Manual testing of these functionalities becomes repetitive, time-consuming, and prone to human error.

To solve this problem, Selenium WebDriver was used for browser automation while Pytest was used for test execution, assertions, parameterization, etc. The strategy included positive testing, negative testing, end-to-end testing, as data-driven testing. Assertions were implemented throughout the framework to validate testing scenerios.

Execution logs confirmed successful validations such as:

- “ASSERT PASSED: Page title contains ‘Myntra’”
- “ASSERT PASSED: Product is present in shopping cart”
- “ASSERT PASSED: Invalid sort option ‘free’ correctly absent from dropdown”

Test Design and Scenario Coverage

The framework was designed using modular automation principles. Separate Page Object classes were created for better maintainability and reusable code structure. Explicit wait methods were implemented to improve synchronization and reduce flaky test failures.

The project covers the following functionalities:

- Homepage validation
- Home & Living menu navigation
- Category navigation
- Product selection
- Add to cart functionality
- Cart validation
- Login and OTP flow
- Sorting validation
- Brand filter validation
- Negative filter validation

Positive scenarios validated workflows such as navigating to categories, applying sorting filters, adding products to cart, and verifying checkout flow. Negative scenarios validated that invalid filters and invalid sorting options were absent on specific pages.

Framework Design

The framework follows a layered architecture with separate folders for tests, features, page objects, utilities, reports, screenshots, and Allure results.

Main framework components:

- `selenium/tests/` → Selenium Pytest test files
- `bdd/features/` → BDD feature files
- `pages/` → Page Object classes
- `utilities/` → Logger, screenshot, config, CSV loader
- `reports/` → Allure reporting data

Reusable helper methods were created for:

- Waiting for URL changes
- Waiting for visible elements
- Waiting for clickable elements
- Title verification
- Screenshot capture

Selenium WebDriver with Pytest

Selenium WebDriver was used to automate browser actions such as clicking buttons, hovering menus, selecting filters, entering input values, switching tabs, and validating UI elements. Different Selenium locators such as XPath, CSS Selectors, Class Name, and Tag Name were used.

Pytest was integrated for:

- Assertions
- Parameterized testing
- Fixtures
- Logging
- Test execution
- Allure reporting

CSV-driven testing was implemented using `pytest.mark.parametrize()` to execute multiple category scenarios dynamically.

BDD with Behave

Behavior Driven Development (BDD) was implemented using Behave and Gherkin feature files. Scenarios were written using Given, When, Then, and And syntax to represent real user behavior.

BDD scenarios included:

- User buys Aromas & Candles product successfully
- Validate category navigation

- Negative validation for invalid filters
- Negative validation for invalid sorting options

BDD improved readability, collaboration, and project documentation.

Test Execution, Reporting and Defect Handling

Tests were executed using Pytest and Behave commands. Allure reporting was integrated for detailed execution reports containing pass/fail status, screenshots, and logs.

Failure handling was implemented using assertions, exception handling, logging, and screenshots. Whenever a test failed, screenshots were captured automatically and attached to Allure reports for debugging purposes.

Logs generated during execution included:

- “TEST FAILED”
- “Screenshot saved”
- “ASSERT PASSED”

These logs improved defect tracking and root cause analysis.